

# URDF/SDF 로봇 모델링 & 조인트 기초

Gazebo · XACRO · robot\_state\_publisher

[cchyun@gmail.com](mailto:cchyun@gmail.com)

# Part 1. URDF 기초 개념

# URDF란 무엇인가?

- URDF(Unified Robot Description Format): 로봇의 물리적 구성을 XML 형식으로 기술하는 표준 파일 포맷
  - 코드에서의 활용
    - 로봇 팔의 위치, 센서 방향 등을 매번 수동 계산하지 않아도 됨
    - `robot_state_publisher` 노드가 URDF를 읽어 TF(Transform) 트리를 자동 생성
  - 좌표계 변환
    - 로봇 중심에서 10cm 앞에 달린 LiDAR가 장애물 감지 시
    - URDF 정보를 바탕으로 장애물의 로봇 기준 위치를 즉시 계산
  - 시뮬레이션과 시각화
    - RViz2에서 3D 시각화
    - Gazebo에서 물리 법칙에 따른 동작 → URDF 필수

- URDF는 계층 구조를 가짐: `robot` 태그 안에 `link`와 `joint`가 포함

```
<robot name="my_first_robot">
  <!-- link: 물리적 부품 정의 -->
  <link name="base_link"> ... </link>
  <link name="left_wheel"> ... </link>

  <!-- joint: 링크 간 연결 관계 정의 -->
  <joint name="left_wheel_joint" type="continuous">
    ...
  </joint>
</robot>
```

- `<robot>` : 전체 로봇 정의를 시작/닫는 최상위 태그, 로봇 이름을 속성으로 가짐
- `<link>` : 물리적 부품(몸체, 바퀴 등) 1개를 정의
- `<joint>` : 두 링크 사이의 연결 관계를 정의

## **Part 2. Link 태그 상세**

- link 태그는 3가지 핵심 하위 속성으로 구성
  - <visual> — "어떻게 보이는가"
    - 색상(material)이나 3D 모델 파일(geometry)을 설정
    - RViz에서 시각화할 때 사용
  - <collision> — "충돌 범위가 어디인가"
    - 물리 엔진이 충돌 계산 시 사용
    - 성능을 위해 visual 보다 단순한 상자/원통형으로 만드는 것이 권장
  - <inertial> — "물리적 질량이 어떠한가"
    - 질량(mass)과 관성 텐서(inertia) 값을 가짐
    - 시뮬레이션에서 실제처럼 움직이게 하는 핵심 정보

```
<link name="base_link">
  <!-- 시각화: RViz에서 보이는 모양 -->
  <visual>
    <geometry><box size="0.4 0.3 0.1"/></geometry>
    <material name="white"/>
  </visual>

  <!-- 충돌: 물리 엔진이 사용하는 단순화된 형태 -->
  <collision>
    <geometry><box size="0.4 0.3 0.1"/></geometry>
  </collision>

  <!-- 관성: 질량과 관성 모멘트 (시뮬레이션 필수) -->
  <inertial>
    <mass value="1.0"/>
    <inertia ixx="0.01" ixy="0" ixz="0"
              iyy="0.01" iyz="0" izz="0.01"/>
  </inertial>
</link>
```

# Part 3. Joint 태그 상세



- joint 태그는 두 link 사이의 관계를 정의하며 트리 구조를 형성
  - <parent> / <child>
    - 어떤 링크가 기준(<parent>)이고, 어떤 링크가 종속되어 움직이는지(<child>) 결정
    - 이 관계가 모여 트리 구조를 형성
  - <origin>
    - 부모 링크로부터 자식 링크의 상대적 위치(<xyz>)와 회전(<rpy>)을 지정
  - <axis>
    - 회전이나 이동이 일어나는 축(예: x, y, z축)을 지정
  - <limit>
    - 관절의 최소/최대 이동 범위와 최대 힘(effort), 속도를 제한

```
<joint name="left_wheel_joint" type="continuous">
  <!-- 부모-자식 관계 정의 -->
  <parent link="base_link"/>
  <child link="left_wheel"/>

  <!-- 부모 기준 자식의 위치(xyz)와 회전(rpy) -->
  <!-- rpy: -1.5708 rad = -90도 회전 (바퀴를 세우기 위해) -->
  <origin xyz="-0.1 0.175 0" rpy="-1.5708 0 0"/>

  <!-- 회전축: z축 기준 회전 -->
  <axis xyz="0 0 1"/>
</joint>
```

- `xyz="-0.1 0.175 0"` : base\_link 중심에서 앞으로 -0.1m, 왼쪽으로 0.175m
- `rpy="-1.5708 0 0"` : Roll -90° → 원통을 세워서 바퀴 방향으로 정렬
- `axis xyz="0 0 1"` : 로컬 z축을 기준으로 회전

## Joint 타입의 종류

| 타입                      | 동작          | 활용 예           |
|-------------------------|-------------|----------------|
| <code>fixed</code>      | 고정 (움직임 없음) | 카메라, LiDAR 마운트 |
| <code>revolute</code>   | 각도 제한 있는 회전 | 로봇 팔 관절, 조향 바퀴 |
| <code>continuous</code> | 무한 회전       | 구동 바퀴          |
| <code>prismatic</code>  | 직선 이동       | 리프트, 슬라이드      |
| <code>floating</code>   | 6자유도 자유 이동  | 드론, 수중 로봇 베이스  |
| <code>planar</code>     | 평면 2D 자유 이동 | 평면 이동 로봇       |

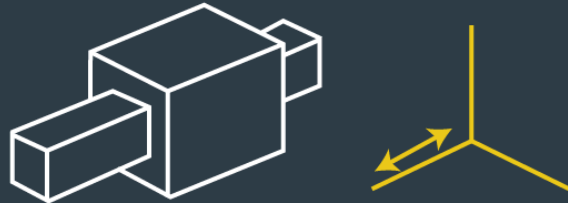
- 입문 수준에서 자주 쓰는 타입: `fixed`, `revolute`, `continuous`
  - `fixed` → 센서 부착
  - `continuous` → 구동 바퀴
  - `revolute` → 로봇 팔, 조향

## Lower Order Pairs

Revolute (1R)



Prismatic (1T)



Screw (1R/T)

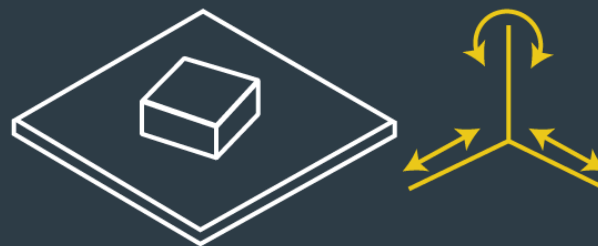


## Higher Order Pairs

Spherical (3R)



Planar (2T, 1R)



Cylindrical (1T, 1R)



R = Rotation, T = Translation

# Part 4. 관성 텐서 & 물리 설정

- Gazebo에서 로봇이 안정적으로 동작하려면 정확한 `inertial` 값이 필수

- 설정 방법

- `<mass value="1.0"/>` : 무게(kg) 설정
- `<inertia>` 안에 `ixx`, `ixy`, `ixz`, `iyy`, `iyz`, `izz` 6개 값 입력

```
<inertial>
  <mass value="1.0"/>
  <inertia ixx="0.01" ixy="0" ixz="0"
            iyy="0.01" iyz="0" izz="0.01"/>
</inertial>
```

- ⚠ 경고: 잘못된 관성 값의 결과
  - 질량을 0으로 설정하거나 관성 값이 너무 작으면
  - 물리 엔진이 수치적으로 불안정해짐
  - 로봇이 격렬하게 진동하거나 하늘로 날아가는 현상 발생

| 값        | 의미                                             |
|----------|------------------------------------------------|
| $i_{xx}$ | x축 중심으로 회전할 때의 저항 (y, z 방향으로 질량이 멀리 퍼져 있을수록 ↑) |
| $i_{yy}$ | y축 중심으로 회전할 때의 저항 (x, z 방향으로 질량이 멀리 퍼져 있을수록 ↑) |
| $i_{zz}$ | z축 중심으로 회전할 때의 저항 (x, y 방향으로 질량이 멀리 퍼져 있을수록 ↑) |
| $i_{xy}$ | x축 회전과 y축 회전이 서로 얼마나 영향을 주는가 (비대칭 물체에서 0 아님)   |
| $i_{xz}$ | x축 회전과 z축 회전의 상호 영향                            |
| $i_{yz}$ | y축 회전과 z축 회전의 상호 영향                            |

# Part 5. URDF 검증 도구



- 코드가 길어지면 오타가 발생하기 쉬움 — 반드시 검증 후 사용
  - ① `check_urdf <파일명>.urdf`
    - XML 문법 오류 여부, 링크 연결이 끊기지 않았는지 확인

```
$ check_urdf my_robot.urdf
robot name is: basic_bot
----- Successfully Parsed XML -----
```

- ② `urdf_to_graphviz <파일명>.urdf`
  - 로봇의 연결 구조를 시각적 그래프(PDF)로 생성
  - 부모-자식 관계가 의도대로 설계되었는지 한눈에 확인

# **Part 6. URDF vs SDF**

- 용도에 따라 URDF와 SDF를 구분하여 사용
  - URDF (Unified Robot Description Format)
    - ROS의 표준 포맷
    - RViz2 시각화와 로봇의 **운동학적 분석**에 최적화
    - 단점: **닫힌 루프(Closed-loop)** 구조 표현 불가
  - SDF (Simulation Description Format)
    - Gazebo 시뮬레이터의 기본 포맷
    - 로봇뿐 아니라 **세계(World), 조명, 마찰력, 물리 엔진 설정** 등 포함
    - 시뮬레이션에 특화된 고급 정보를 표현 가능
    - Gazebo는 내부적으로 URDF를 **SDF로 변환**하여 실행
  - **실용 팁**: 로봇 모델 기술은 URDF/XACRO, 시뮬레이션 세계 구성은 SDF

# Part 7. 입문자가 자주 하는 실수

- 시뮬레이션 안정성을 위해 아주 작은 부품이라도 0이 아닌 물리값을 주어야 함

- ❌ 잘못된 코드

```
<inertial>
  <mass value="0"/>    <!-- 질량 0 → 물리 엔진 불안정 -->
  <inertia ixx="0" ixy="0" ixz="0" iyy="0" iyz="0" izz="0"/>
</inertial>
```

- ✅ 올바른 코드

```
<inertial>
  <mass value="0.1"/>    <!-- 작더라도 0이 아닌 값 필요 -->
  <inertia ixx="0.001" ixy="0" ixz="0"
            iyy="0.001" iyz="0" izz="0.001"/>
</inertial>
```

- 로봇을 바닥( $Z=0$ )과 정확히 겹치게 생성하면 바퀴가 바닥을 뚫고 시뮬레이션이 튕김
  - ✗ 잘못된 방식:  $Z=0$ 에 생성
    - 바퀴가 바닥면과 겹쳐 물리 엔진 충돌 → 튕김 또는 폭발
  - ✔ 올바른 방식: 약간 공중에 띄워 생성
    - 0.1 ~ 0.5m 높이에서 생성 후 자연스럽게 바닥으로 내려앉게 유도

```
# Gazebo spawn 시 Z 위치 설정 예시
spawn_entity = Node(
    package='gazebo_ros',
    executable='spawn_entity.py',
    arguments=['-topic', 'robot_description',
               '-entity', 'my_robot',
               '-z', '0.1'] # Z축으로 0.1m 띄워서 생성
)
```

- ROS 표준 좌표계 방향을 반드시 준수해야 함

- ROS 표준 좌표계 (REP-103)

| 축  | 색상                                                                                    | 방향         |
|----|---------------------------------------------------------------------------------------|------------|
| X축 |  빨간색 | 앞(Forward) |
| Y축 |  초록색 | 왼쪽(Left)   |
| Z축 |  파란색 | 위(Up)      |

- 이 규칙을 어기면
  - 로봇이 옆으로 이동하거나 거꾸로 움직이는 혼란 발생
  - 센서 데이터가 엉뚱한 방향을 가리킴
- RViz2에서 확인: 축 색상을 보고 로봇의 앞/뒤/좌/우 방향을 검증

# Part 8. XACRO — XML 매크로



- 순수 URDF의 반복 코드 문제를 해결하기 위해 XACRO 도입
  - 문제: 순수 URDF는 단순 XML → 중복 코드 불가피
    - 동일한 모양의 바퀴 4개 → 동일 XML 블록을 4번 반복 작성
    - 코드가 길어지고, 수정 시 실수 발생 위험 증가
  - XACRO(XML Macros)의 해결책
    - 변수와 함수처럼 작동하는 매크로를 통해 코드 재사용
    - 코드 가독성 향상 및 유지보수 자동화

순수 URDF: left\_wheel 블록 + right\_wheel 블록 (중복)

↓

XACRO: wheel 매크로 정의 1번 + 매크로 호출 2번

- ① property (변수 정의)

- `${property_name}` 구문으로 전반에서 재사용할 상수 정의

```
<xacro:property name="wheel_radius" value="0.05"/>
<xacro:property name="wheel_length" value="0.04"/>
<xacro:property name="chassis_mass" value="1.0"/>
```

```
<!-- 사용: ${wheel_radius}로 참조 -->
```

```
<geometry><cylinder radius="${wheel_radius}" length="${wheel_length}"/></geometry>
```

- ② macro (재사용 가능한 블록)
  - 반복 구성 요소를 매개변수로 호출

```
<xacro:macro name="wheel" params="prefix reflect">
  <link name="${prefix}_wheel">
    <geometry><cylinder radius="${wheel_radius}" length="${wheel_length}" /></geometry>
  </link>
</xacro:macro>

<!-- 호출 -->
<xacro:wheel prefix="left" reflect="1" />
<xacro:wheel prefix="right" reflect="-1"/>
```

- ③ include (다른 xacro 파일 포함)
  - 로봇 모델을 여러 파일로 분할하여 관리

```
<!-- 센서, 구동부, 본체를 각각 파일로 관리 -->  
<xacro:include filename="$(find my_robot)/urdf/sensors.xacro"/>  
<xacro:include filename="$(find my_robot)/urdf/drive.xacro"/>
```

- ④ 조건문 (xacro:if)
  - 특정 조건에 따라 URDF의 일부를 포함/제외

```
<!-- use_sim=true일 때만 Gazebo 센서 플러그인 포함 -->  
<xacro:if value="${use_sim}">  
  <gazebo reference="lidar_link">  
    <sensor type="ray" name="lidar"> ... </sensor>  
  </gazebo>  
</xacro:if>
```

- XACRO 파일은 실행 전에 순수 URDF로 변환되어야 함

- 터미널에서 직접 변환

```
# XACRO → URDF 파일 생성
```

```
ros2 run xacro xacro src/my_robot_description/urdf/turtlebot.xacro \  
  > src/my_robot_description/urdf/turtlebot_from_xacro.urdf
```

```
# 변환 결과 검증
```

```
check_urdf src/my_robot_description/urdf/turtlebot_from_xacro.urdf
```

- 런치 파일 내에서 동적 변환 (권장 방식)

- 별도의 변환 파일 생성 없이 런타임에 XACRO 출력을 직접 파라미터로 전달

```
from launch.substitutions import Command
```

```
# xacro 명령어 실행 결과를 robot_description으로 바로 전달
```

```
robot_description_content = Command(['xacro ', xacro_file])
```

# Part 9. 실습 — XACRO로 TurtleBot 모 델링

- **목표:** XACRO로 2륜 차동 구동 로봇(TurtleBot3 Burger 스타일)을 모델링하고 RViz2로 시각화하기

- **사전 준비**

```
# 워크스페이스 루트로 이동
cd ~/Workspace/ros2-ws

# 빌드 도구 설치
sudo apt install ros-humble-xacro ros-humble-joint-state-publisher-gui
```

- **완성 구조**

- `base_footprint` → `base_link` → `wheel_left` / `wheel_right` / `caster_back`
- 센서: `imu_link`, `base_scan`, `camera_link`

## 9-1. 패키지 생성 및 디렉토리 구조

```
cd ~/Workspace/ros2-ws/src
ros2 pkg create my_robot_description \
  --build-type ament_cmake \
  --dependencies urdf xacro

cd my_robot_description
mkdir -p urdf launch rviz
```

- 디렉토리 구조

```
my_robot_description/
├── CMakeLists.txt
├── package.xml
├── urdf/
├── launch/
└── rviz/
```



## 9-1. 패키지 생성 및 디렉토리 구조

- CMakeLists.txt 수정 — URDF 파일을 설치 디렉토리에 포함

```
cmake_minimum_required(VERSION 3.8)
project(my_robot_description)

find_package(ament_cmake REQUIRED)

install(
  DIRECTORY urdf launch rviz
  DESTINATION share/${PROJECT_NAME}
)

ament_package()
```

## 9-2. XACRO 기본 구조 — 베이스 링크 (1/2)

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="turtlebot">

  <!-- ===== MATERIALS ===== -->
  <material name="black">
    <color rgba="0.0 0.0 0.0 1.0"/>
  </material>
  <material name="dark">
    <color rgba="0.3 0.3 0.3 1.0"/>
  </material>
  <material name="light_grey">
    <color rgba="0.75 0.75 0.75 1.0"/>
  </material>
  <material name="blue">
    <color rgba="0.0 0.0 0.8 1.0"/>
  </material>

  <!-- base_footprint: 로봇의 가상 바닥 위치 -->
  <link name="base_footprint"/>
</robot>
```

- `<material>`: 재사용 가능한 색상 정의
- `base_footprint`: 로봇의 2D 위치 추적 기준점 (가상 링크)

## 9-2. XACRO 기본 구조 — 베이스 링크 (2/2)

```
<joint name="base_joint" type="fixed">
  <parent link="base_footprint"/>
  <child link="base_link"/>
  <origin xyz="0.0 0.0 0.010" rpy="0 0 0"/>
</joint>

<!-- base_link: 실제 로봇 몸체 (Box 형태) -->
<link name="base_link">
  <visual>
    <origin xyz="-0.032 0 0.070" rpy="0 0 0"/>
    <geometry>
      <box size="0.140 0.140 0.143"/>
    </geometry>
    <material name="light_grey"/>
  </visual>
  <collision>
    <origin xyz="-0.032 0 0.070" rpy="0 0 0"/>
    <geometry>
      <box size="0.140 0.140 0.143"/>
    </geometry>
  </collision>
  <inertial>
    <origin xyz="-0.032 0 0" rpy="0 0 0"/>
    <mass value="8.2573504e-01"/>
    <inertia ixx="2.2124416e-03" ixy="-1.2294101e-05" ixz="3.4938785e-05"
      iyy="2.1193702e-03" iyz="-5.0120904e-06" izz="2.0064271e-03"/>
  </inertial>
</link>
```

## 9-3. 휠 매크로 추가

```
<!-- ===== WHEEL MACRO ===== -->
<xacro:macro name="wheel" params="prefix y_offset">
  <link name="${prefix}_link">
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
        <cylinder length="0.018" radius="0.033"/>
      </geometry>
      <material name="black"/>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
        <cylinder length="0.018" radius="0.033"/>
      </geometry>
    </collision>
    <inertial>
      <origin xyz="0 0 0"/>
      <mass value="2.8498940e-02"/>
      <inertia ixx="1.1175580e-05" ixy="-4.2369783e-11" ixz="-5.9381719e-09"
        iyy="1.1192413e-05" iyz="-1.4400107e-11" izz="2.0712558e-05"/>
    </inertial>
  </link>

  <joint name="${prefix}_joint" type="continuous">
    <parent link="base_link"/>
    <child link="${prefix}_link"/>
    <origin xyz="0.0 ${y_offset} 0.023" rpy="-1.57 0 0"/>
    <axis xyz="0 0 1"/>
  </joint>
</xacro:macro>

<!-- ===== WHEELS ===== -->
<xacro:wheel prefix="wheel_left" y_offset="0.08"/>
<xacro:wheel prefix="wheel_right" y_offset="-0.08"/>
```

- `rpy="-1.57 0 0"` (joint): 조인트 회전축을 Z축으로 정렬

## 9-3. 캐스터 매크로 추가

```
<!-- ===== CASTER MACRO ===== -->
<xacro:macro name="caster" params="prefix x_offset">
  <joint name="caster_${prefix}_joint" type="fixed">
    <parent link="base_link"/>
    <child link="caster_${prefix}_link"/>
    <origin xyz="${x_offset} 0 -0.004" rpy="-1.57 0 0"/>
  </joint>

  <link name="caster_${prefix}_link">
    <visual>
      <origin xyz="0 0.001 0" rpy="0 0 0"/>
      <geometry>
        <sphere radius="0.0075"/>
      </geometry>
      <material name="light_grey"/>
    </visual>
    <collision>
      <origin xyz="0 0.001 0" rpy="0 0 0"/>
      <geometry>
        <sphere radius="0.005"/>
      </geometry>
    </collision>
    <inertial>
      <origin xyz="0 0 0"/>
      <mass value="0.005"/>
      <inertia ixx="0.001" ixy="0.0" ixz="0.0"
        iyy="0.001" iyz="0.0" izz="0.001"/>
    </inertial>
  </link>
</xacro:macro>

<!-- ===== CASTERS ===== -->
<xacro:caster prefix="back" x_offset="-0.081"/>
```

- 캐스터는 Sphere 형태의 visual/collision을 가지는 보조 지지대

## 9-3. 빌드 및 XACRO 변환

```
cd ~/Workspace/ros2-ws
colcon build --packages-select my_robot_description
source install/setup.bash

# XACRO → URDF 변환
ros2 run xacro xacro \
  ./src/my_robot_description/urdf/turtlebot.xacro \
  > ./src/my_robot_description/urdf/turtlebot_from_xacro.urdf

# 변환 결과 검증
check_urdf ./src/my_robot_description/urdf/turtlebot_from_xacro.urdf

# robot_state_publisher 실행
ros2 run robot_state_publisher robot_state_publisher \
  ./src/my_robot_description/urdf/turtlebot_from_xacro.urdf

# 터미널 3: 조인트 수동 제어 GUI 실행
ros2 run joint_state_publisher_gui joint_state_publisher_gui
```

## 9-4. 센서 프레임 추가 — IMU & LDS & 카메라

```
<!-- ===== IMU ===== -->
<joint name="imu_joint" type="fixed">
  <parent link="base_link"/>
  <child link="imu_link"/>
  <origin xyz="-0.032 0 0.068" rpy="0 0 0"/>
</joint>
<link name="imu_link"/>
```

- `imu_link`: 관성 측정 장치(IMU)의 좌표계 (빈 링크 — Gazebo 플러그인에서 참조)

## 9-4. 센서 프레임 추가 — LDS (LiDAR)

```
<!-- ===== BASE SCAN (LDS) ===== -->
<joint name="scan_joint" type="fixed">
  <parent link="base_link"/>
  <child link="base_scan"/>
  <origin xyz="-0.04 0 0.172" rpy="0 0 0"/>
</joint>

<link name="base_scan">
  <visual>
    <origin xyz="0 0 0.0" rpy="0 0 0"/>
    <geometry>
      <cylinder length="0.0315" radius="0.045"/>
    </geometry>
    <material name="dark"/>
  </visual>
  <collision>
    <origin xyz="0.015 0 -0.0065" rpy="0 0 0"/>
    <geometry>
      <cylinder length="0.0315" radius="0.045"/>
    </geometry>
  </collision>
  <inertial>
    <mass value="0.114"/>
    <origin xyz="0 0 0"/>
    <inertia ixx="0.001" ixy="0.0" ixz="0.0"
      iyy="0.001" iyz="0.0" izz="0.001"/>
  </inertial>
</link>
```

- **base\_scan**: Laser Distance Sensor (LiDAR)의 시각적/물리적 모델



## 9-4. 센서 프레임 추가 — 카메라

```
<!-- ===== CAMERA ===== -->
<joint name="camera_joint" type="fixed">
  <parent link="base_link"/>
  <child link="camera_link"/>
  <origin xyz="0.028 0 0.1515" rpy="0 0 0"/>
</joint>

<link name="camera_link">
  <visual>
    <geometry>
      <box size="0.02 0.02 0.02"/>
    </geometry>
    <material name="blue"/>
  </visual>
  <collision>
    <geometry>
      <box size="0.02 0.05 0.02"/>
    </geometry>
  </collision>
  <inertial>
    <mass value="0.001"/>
    <inertia ixx="0.0001" ixy="0" ixz="0"
      iyy="0.0001" iyz="0"
      izz="0.0001"/>
  </inertial>
</link>
```

## 8-5. 빌드 및 robot\_state\_publisher 실행

```
cd ~/Workspace/ros2-ws
colcon build --packages-select my_robot_description
source install/setup.bash

# XACRO → URDF 변환
ros2 run xacro xacro \
  ./src/my_robot_description/urdf/turtlebot.xacro \
  > ./src/my_robot_description/urdf/turtlebot.urdf

# 변환된 URDF로 robot_state_publisher 실행
ros2 run robot_state_publisher robot_state_publisher \
  ./src/my_robot_description/urdf/turtlebot.urdf

# 터미널 3: 조인트 수동 제어 GUI 실행
ros2 run joint_state_publisher_gui joint_state_publisher_gui
```

# **Part 10. robot\_state\_publisher & Launch 통합**

- 로봇의 기하학적 구조와 현재 조인트 상태를 결합하여 TF 트리를 관리
  - TF2 트리 자동 발행
    - URDF의 `joint` 정의를 읽어 각 `link` 간의 정적 변환(fixed joint)을 TF로 발행
  - `/joint_states` 토픽 구독
    - 모터 등에 의해 움직이는 조인트(revolute, continuous)의 현재 각도/위치 정보 수신
    - 이를 바탕으로 움직이는 부분의 TF 변환을 실시간 계산 및 업데이트
- 전체 흐름

URDF 파일

↓ (robot\_description 파라미터로 전달)

robot\_state\_publisher

↑ `/joint_states` (모터 각도 정보)

↓ `/tf`, `/tf_static` (전체 좌표 변환 트리)

RViz2, Navigation Stack 등

- 한 번의 명령으로 로봇 모델 + 조인트 제어 + RViz2 전체 실행

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration, Command
from launch_ros.actions import Node

def generate_launch_description():
    pkg_share = get_package_share_directory('my_robot_description')
    xacro_file = os.path.join(pkg_share, 'urdf', 'turtlebot.xacro')
    rviz_config_file = os.path.join(pkg_share, 'rviz', 'turtlebot.rviz')

    # XACRO → URDF 내용 변환 (런타임 실행)
    robot_description_content = Command(['xacro ', xacro_file])

    return LaunchDescription([
        DeclareLaunchArgument(
            'use_sim_time', default_value='false',
            description='Use simulation clock if true'),
```

```
Node(  
    package='robot_state_publisher',  
    executable='robot_state_publisher',  
    parameters=[  
        'robot_description': robot_description_content,  
        'use_sim_time': LaunchConfiguration('use_sim_time')  
    ]  
)  
Node(  
    package='joint_state_publisher_gui',  
    executable='joint_state_publisher_gui'  
)  
Node(  
    package='rviz2',  
    executable='rviz2',  
    arguments=['-d', rviz_config_file],  
    parameters=[  
        'use_sim_time': LaunchConfiguration('use_sim_time')  
    ]  
)  
])
```



```
<!-- my_robot_description/package.xml -->  
<depend>robot_state_publisher</depend>  
<depend>joint_state_publisher_gui</depend>  
<depend>rviz2</depend>  
<depend>xacro</depend>
```

```
# 빌드 후 한 줄로 실행  
colcon build --packages-select my_robot_description  
source install/setup.bash  
ros2 launch my_robot_description display.launch.py
```